

Virtual Machine per il computer Hack

Codice Virtual Machine

```
push constant 10
call inc 1
label LOOP
goto LOOP

function inc 0
push argument 0
push constant 1
add
return
```



traduttore

Codice assembly

```
//bootstrap
@256
D=A
@SP
M=D

//push constant 10
@10
D=A
@SP
A=M
M=D
@SP
M=M+1

//call inc 1
@1000000000000000
D=A
@SP
A=M
M=D
@SP
M=M+1
...
```

Virtual machine

Una virtual machine è un modello astratto di calcolo, e non fa riferimento ad una specifica architettura.

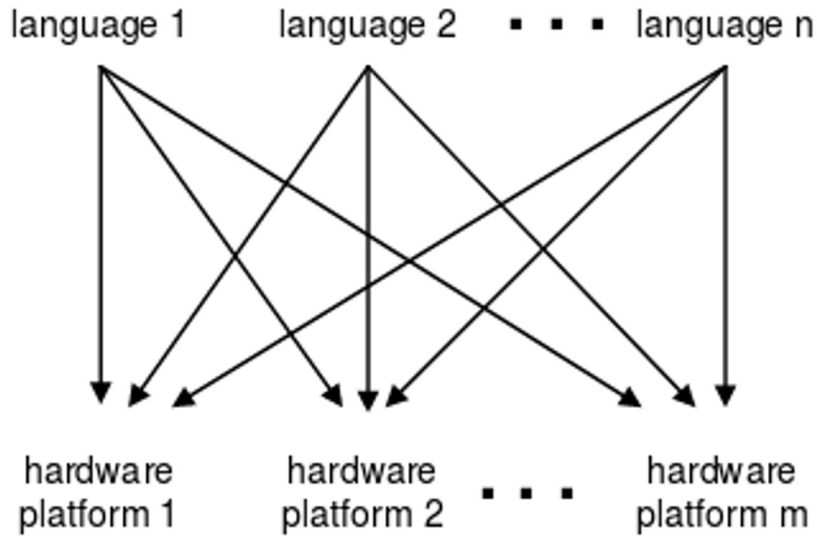
Costituisce un utile livello intermedio tra un linguaggio ad alto livello ed il linguaggio assembly.

Usata nel modello di compilazione **a 2 livelli**.

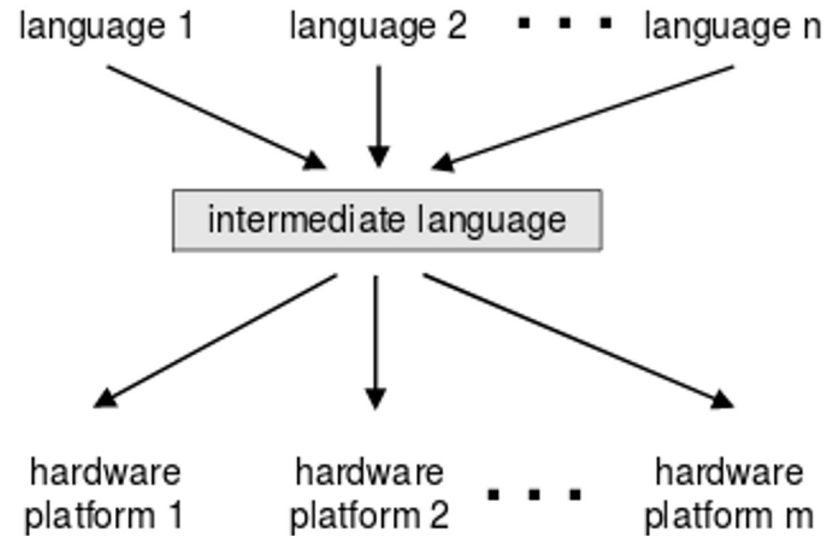
Ad esempio, la JVM (Java Virtual Machine) di Java, basata su stack, oppure BEAM di Erlang, basata su registri.

Modelli di compilazione

Compilazione diretta



Compilazione a 2 livelli



Compilazione a 2 livelli

- Primo livello: si compila il codice in un linguaggio intermedio che non dipende dall'architettura fisica. Quindi dipende solo dai dettagli del linguaggio **sorgente**;
- Secondo livello: dal linguaggio intermedio a linguaggio **target**. Dipende solo dai dettagli del linguaggio target.

Virtual Machine

Nel nostro caso, il livello intermedio è costituito dalla Virtual Machine Hack la quale, come ogni virtual machine, è associata con un proprio linguaggio.

Programmi in tale linguaggio sono il risultato del primo livello di compilazione.

Occorre compilare i programmi in questo linguaggio per VM per ottenere codice ISA (programmi in assembly simbolico o codice macchina).

Il nostro linguaggio per Virtual Machine (per Hack)

Operazioni logico / booleane:

add	(somma)
sub	(sottrazione)
neg	(negazione aritmetica)
eq	(test di uguaglianza)
gt	(test gt)
lt	(test lt)
and	(and bit-a-bit)
or	(or bit-a-bit)
not	(not bit-a-bit)

Flusso di controllo:

label L	(dich. di etichetta)
goto L	(salto incondizionato)
if-goto L	(salto condizionato)

Chiamate a funzione:

function F k	(dichiarazione)
call F k	(chiamata a funzione)
return	(istruzione di ritorno)

Accesso alla memoria (usando uno stack):

Pop x	(pop della variabile x dallo stack, ovvero: il valore in cima è il nuovo valore di x)
Push y	(push di variabile o costante y sullo stack: il valore viene copiato in cima)

Il nostro linguaggio per Virtual Machine

Come procediamo:

- Guardiamo uno ad uno i possibili tipi di istruzioni, definendone la semantica sul modello di macchina virtuale Hack (cioè definendo come manipolano lo stack e la memoria)
- Poi vedremo esempi gradualmente di come possiamo scrivere programmi in questo linguaggio per VM Hack
- Infine vediamo alcuni dettagli relativi alla traduzione di un programma per VM Hack in assembly Hack.

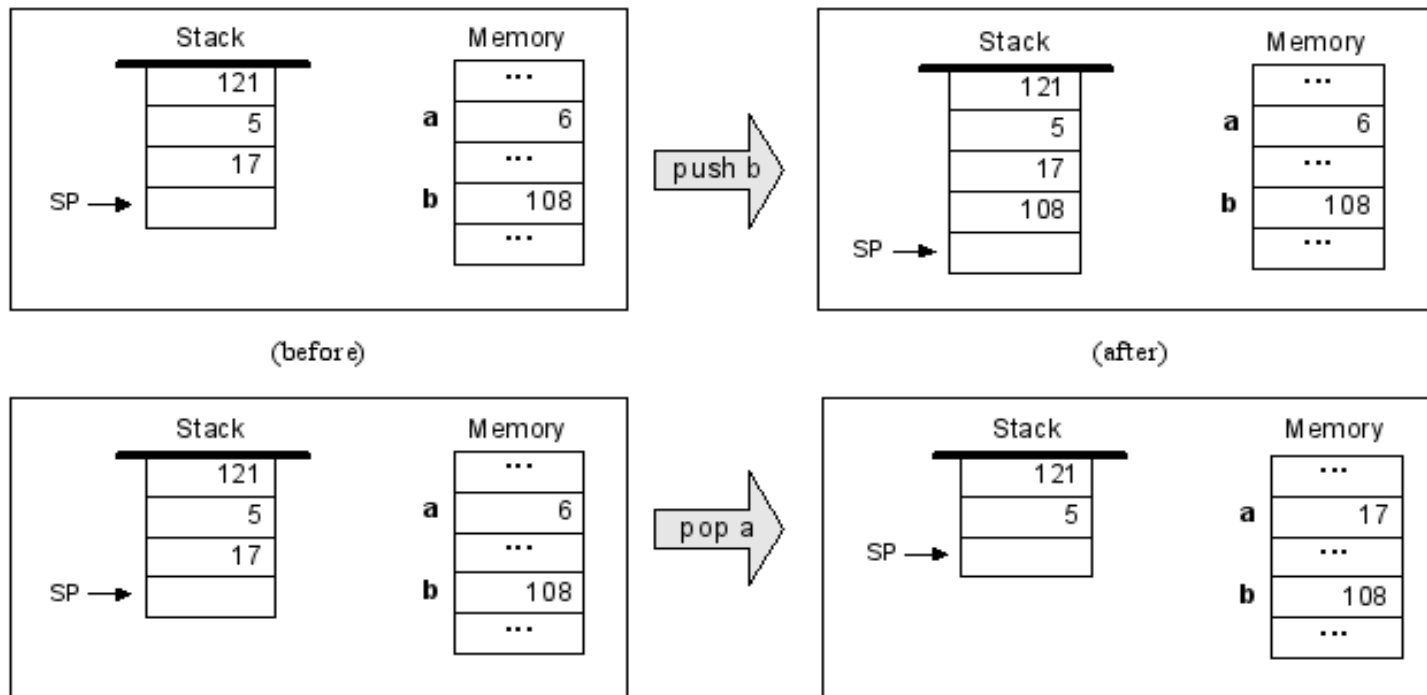
La difficoltà di questa traduzione/compilazione è che il programma ISA dovrà occuparsi esplicitamente della gestione della memoria, che invece al livello della VM viene dato come comportamento automatico della macchina virtuale (perché il suo funzionamento è definito in astratto).

(la VM può essere emulata dal VM Emulator, che non rappresenta una macchina fisica ma un modello astratto di calcolo, e viene emulato appunto usando internamente i chip del computer Hack per l'effettiva esecuzione. Nell'ultimo progetto bisogna infatti programmare un compilatore che faccia esattamente la stessa cosa: compilare da programmi per VM a codice assembly Hack).

Comandi di accesso in memoria: push e pop

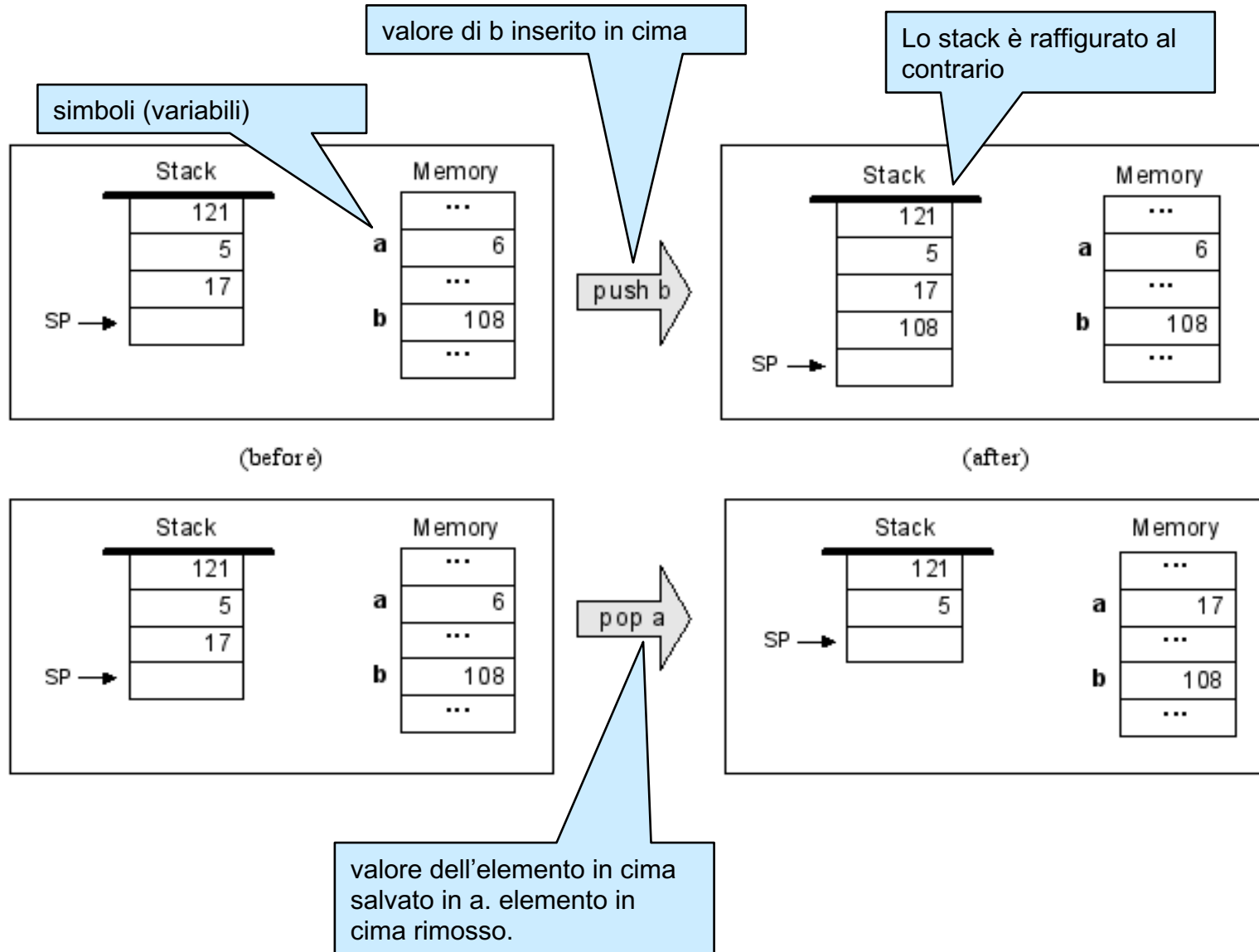
Il modello astratto di calcolo della VM è costituito **da uno stack** (che permette operazioni di push e pop ed è usato per supportare la computazione), in aggiunta ad **una memoria** a registri (locazioni con indirizzi).

Un unico tipo di dato a 16 bit che può essere usato per interi, booleani, puntatori.



Vedremo fra qualche slide che in realtà la memoria è leggermente più complessa.

Operazioni di base sullo stack: push e pop



Operazioni aritmetico-logiche sullo stack

Oltre a push e pop, la VM permette di eseguire le tipiche operazioni aritmetico-logiche (unarie o binarie, cioè con uno o due argomenti). Queste sono le operazioni viste prima: somma, sottrazione, negazione, etc.

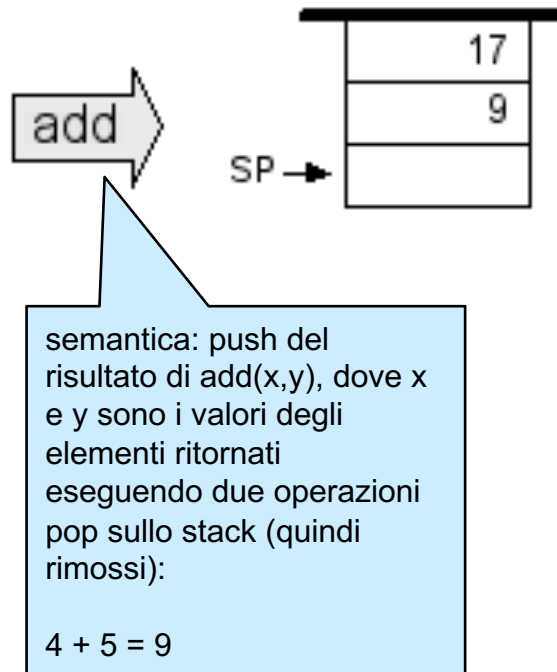
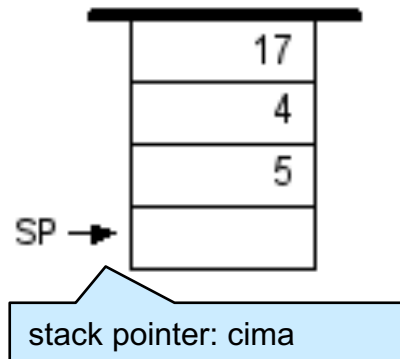
In generale, per eseguire una operazione aritmetico-logica binaria $f(x,y)$ si esegue:

- **Pop** dei valori x,y dalla cima dello stack: questi saranno gli operandi della funzione f ;
- Si calcola il valore della funzione $f(x,y)$ specificata;
- **Push** del risultato in cima allo stack.

Effetto: gli operandi sono rimpiazzati (in cima allo stack) dal risultato.

Tutte le operazioni aritmetico-logiche si comportano in questo modo. Nel caso di operazioni unarie, come $\text{neg}(x)$, ci effettua una sola operazione di *pop*.

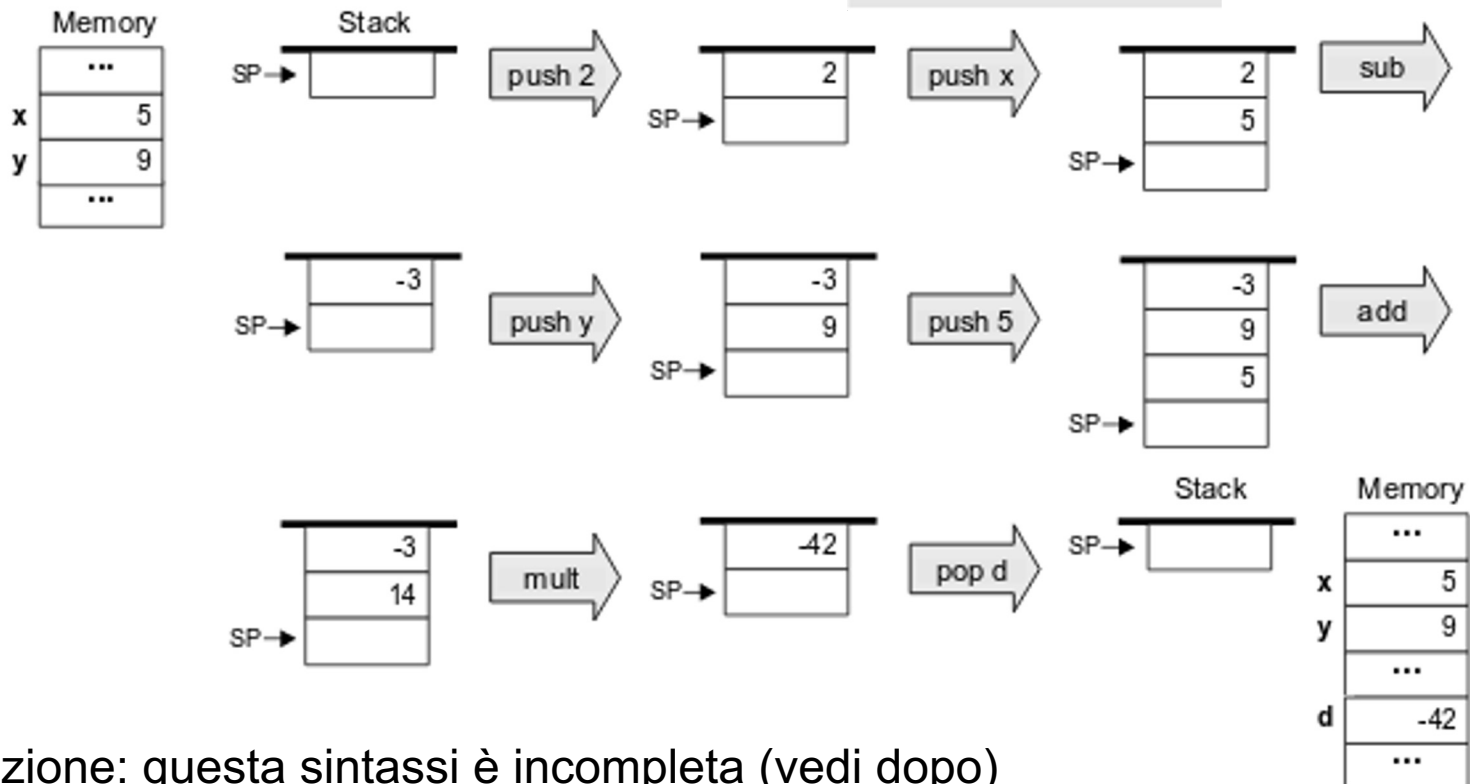
Operazioni sullo stack: esempio



Valutazione di una espressione **aritmetica** (cf. notazione polacca)

Per comodità assumiamo l'esistenza di una operazione "mult" che nel nostro caso non è presente (definiremo più avanti in un esempio una funzione "mult" per la moltiplicazione)

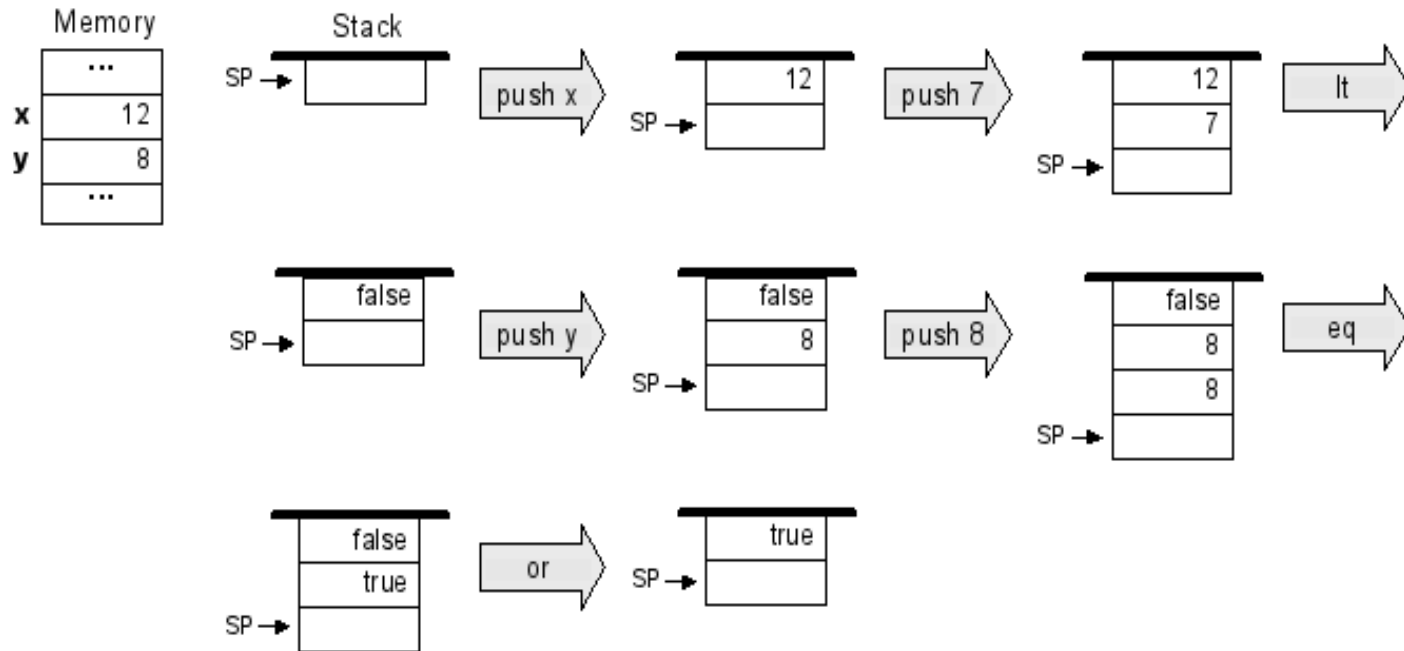
```
// d=(2-x)*(y+5)
push 2
push x
sub
push y
push 5
add
mult
pop d
```



Attenzione: questa sintassi è incompleta (vedi dopo)

Valutazione di una espressione **booleana**

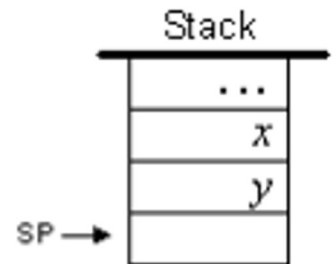
```
// if (x<7) or (y=8)
push x
push 7
lt
push y
push 8
eq
or
```



Attenzione: questa sintassi è incompleta (vedi dopo)

Sommario delle operazioni aritmetico-logiche

Command	Return value (after popping the operand/s)	Comment
add	$x + y$	Integer addition (2's complement)
sub	$x - y$	Integer subtraction (2's complement)
neg	$-y$	Arithmetic negation (2's complement)
eq	true if $x = y$ and false otherwise	Equality
gt	true if $x > y$ and false otherwise	Greater than
lt	true if $x < y$ and false otherwise	Less than
and	$x \text{ And } y$	Bit-wise
or	$x \text{ Or } y$	Bit-wise
not	$\text{Not } y$	Bit-wise



And, Or e Not, quando applicati a valori booleani, rappresentano l'and / or / negazione logici: true è rappresentato come -1 (in complemento a 2: 16 '1') e false come 0.

Segmenti di memoria

La memoria della VM Hack è **segmentata**.

Similmente a quanto studiato in generale, l'adozione di segmenti distinti permette di dividere logicamente la memoria a seconda di scopi, caratteristiche, dimensioni, ruoli, etc.

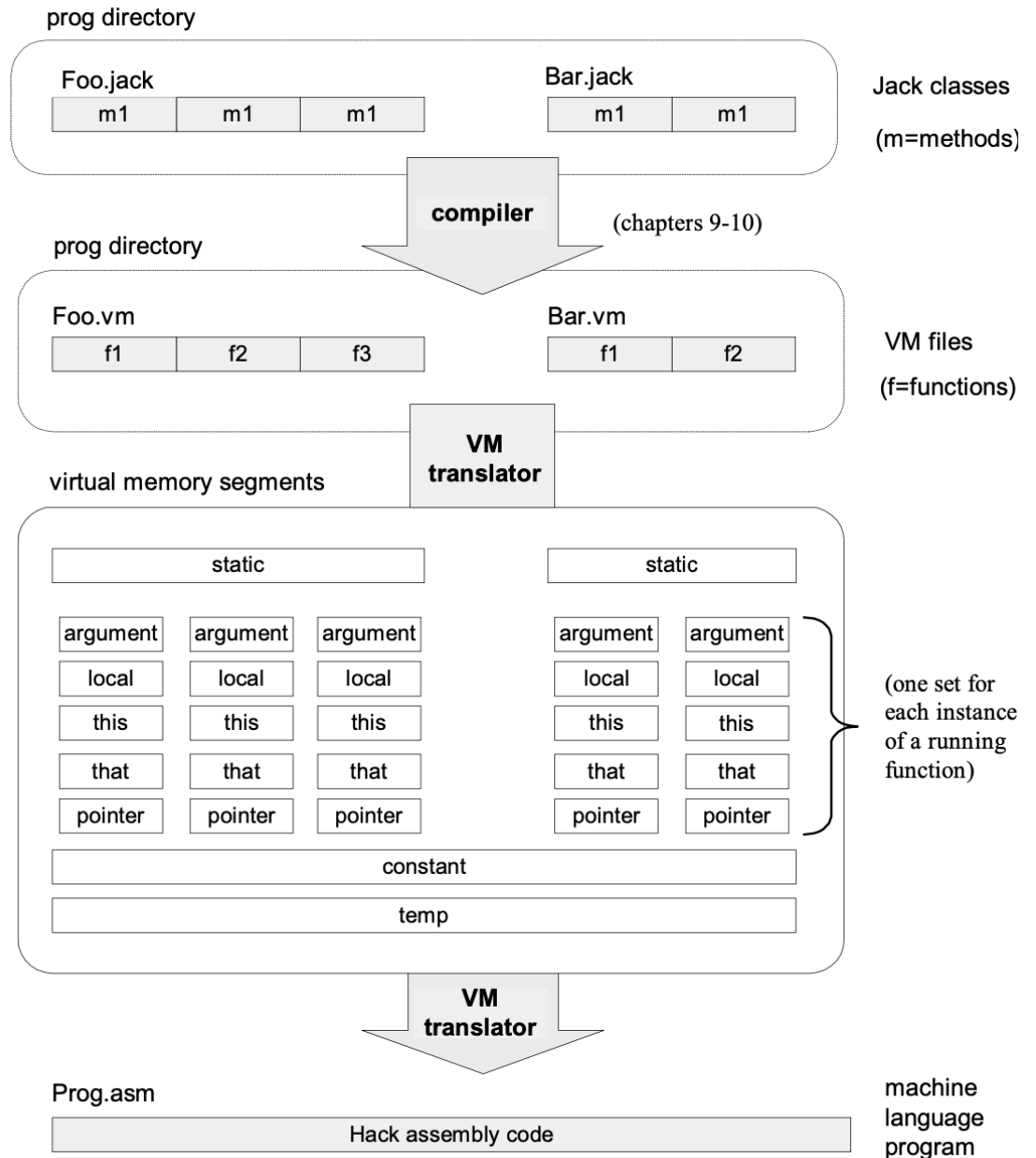
In questo caso, ogni segmento si contraddistingue per il tipo di dato lì memorizzato: argomenti e variabili locali (nel caso di chiamate a funzione), costanti, variabili globali, etc.

La sintassi intuitiva utilizzata finora **va quindi estesa**, menzionando esplicitamente i segmenti su cui operiamo.

(cf. quanto detto nella teoria: la segmentazione della memoria non è trasparente ai programmi)

Segmenti di memoria

Raffigurazione dei due livelli di compilazione nel caso di VM Hack. Non studieremo il linguaggio di alto livello (file .jack).



Segmenti di memoria

Per implementare **le chiamate a funzioni**, ogni funzione in esecuzione avrà due propri segmenti specifici che vengono creati e distrutti prima e dopo l'esecuzione:

- “argument” per gli **argomenti** (parametri di invocazione)
- “local” per le **variabili locali** alla funzione

Confronta quanto studiato sulle chiamate a procedura.

Inoltre, esiste un segmento “static” condiviso da tutte le funzioni, usato per mantenere le **variabili globali**.

Per comodità, si assume anche l'esistenza di un segmento fittizio “constant” da cui si può solo leggere, usato per le **costanti** (cioè un segmento dove all'indirizzo 0 è scritto '0', all'indirizzo 1 è scritto '1', etc). Il motivo di questa scelta è che in questo modo non c'è bisogno di un modo ad-hoc per introdurre costanti nel programma: le leggiamo dal segmento.

Comandi di accesso alla memoria

confronta con indirizzi
(n,k) nelle slide sulla
segmentazione

Accesso ad un segmento in memoria:

```
pop segment addr      // pop dalla cima dello stack, salvando il valore in (segment,addr)
push segment addr     // push in cima dello stack del valore all'indirizzo (segment,addr)
```

Dove *segment* può essere: `static`, `local`, `argument`, `constant`

Al posto di *addr* si indica:

- L'indirizzo di interesse (per `local` e `argument`)
- La costante numerica di interesse (per `constant`)
- Per `static` si usa un valore numerico `n` che viene tradotto (in assembly) nel simbolo `nomeFile.n` assumendo che il file corrente sia `nomeFile.vm`

Esempio:

```
push local 0          // push sullo stack di local[0]
push constant 1       // push sullo stack di constant[1] (quindi costante 1)
sub                   // sottrazione
pop local 0           // pop del valore in cima nella locazione local[0]
```

VMEmulator (per eseguire codice VM, anche disponibile come IDE web)

The screenshot shows the Virtual Machine Emulator (1.4b3) interface. The title bar indicates the file path is G:\examples\add. The menu bar includes File, View, Run, and Help. The toolbar contains icons for file operations and execution controls, along with 'Animate' (Program flow, Script, Decimal) and 'View' (Slow, Fast) options. The main window is divided into several panels:

- Program:** A list of instructions. The instruction at index 11, 'add', is highlighted in yellow and labeled 'VM code'.
- Static:** A table for static variables.
- Local:** A table for local variables. It is labeled 'virtual memory segments'.
- Argument:** A table for arguments.
- This:** A table for 'this' pointers.
- That:** A table for 'that' pointers.
- Temp:** A table for temporary variables.
- global stack:** A table for the global stack. It is labeled 'global stack'.
- host RAM:** A table for host RAM. It is labeled 'host RAM'.
- Stack:** A table for the stack. It is labeled 'working stack'.
- Call Stack:** A table for the call stack.
- Repeat:** A code block containing 'repeat { vmstep; }'. It is labeled 'default test script'.

The 'Repeat' block is highlighted in yellow. The 'host RAM' table shows a range from 0 to 271, with a note '(the RAM is not part of the VM)' pointing to the range. The 'global stack' table shows a range from 264 to 278.

Nota: codice su queste slide ed il VM Emulator

Per essere eseguibili su VM Emulator questi programmi richiedono di inizializzare i segmenti utilizzati.

In queste slide (e negli esercizi, anche d'esame) assumeremo che l'inizializzazione sia stata già fatta. Su virtuale è presente un semplice file test.tst che mostra come inizializzare i segmenti per fare pratica di programmazione per VM usando l'emulatore (ma il file non esegue nessun test: si limita ad inizializzare alcune locazioni ed ad avviare il programma).

In pratica, l'inizializzazione consiste nel settare l'indirizzo di memoria da dove comincia ciascuno dei segmenti necessari. Si tratta di indirizzi fisici Hack. LCL (segmento local) va in RAM[1] e ARG (segmento argument) in RAM[2]. SP va inizializzato a 256 in RAM[0].

Nel caso il programma sia una funzione (vedi tra qualche slide), per poterlo testare (senza eseguire davvero la CALL) occorre inizializzare anche le locazioni necessarie a partire da RAM[RAM[1]] e RAM[RAM[2]] (come se stessimo allocando un record di attivazione).

Quando invece eseguiamo codice che effettivamente chiama una procedura (cioè non stiamo testando solo la funzione per fare pratica o debug), allora i segmenti local e argument saranno inizializzati dall'emulatore.

Esempio: semplice espressione aritmetica

Si scriva codice per la VM Hack che calcola il valore della seguente espressione aritmetica. Notare che non ci serve alcun segmento a parte constant (che è solo di lettura, e non va inizializzato).

```
((5-1) + (4+2)) > (7+2)
```

```
push constant 5  
push constant 1  
sub  
push constant 4  
push constant 2  
add  
add  
push constant 7  
push constant 2  
add  
gt
```

Esempio: semplice espressione booleana

Si scriva codice per la VM Hack che calcola il valore della seguente espressione booleana, assumendo che i segmenti siano già inizializzati e che le variabili x e y siano salvate in `argument[0]` e `argument[1]` (cioè che siano gli argomenti di una chiamata a funzione `f(x,y)` il cui corpo stiamo ora implementando... vedi dopo).

```
x = ((y-1) > (x+2)) OR (x == 7)
```

```
push argument 1
push constant 1
sub
push argument 0
push constant 2
add
gt                // ora in cima abbiamo un boolean
push argument 0
push constant 7
eq
or
pop argument 0    // aggiorniamo il valore dell'argomento x in argument[0]
```

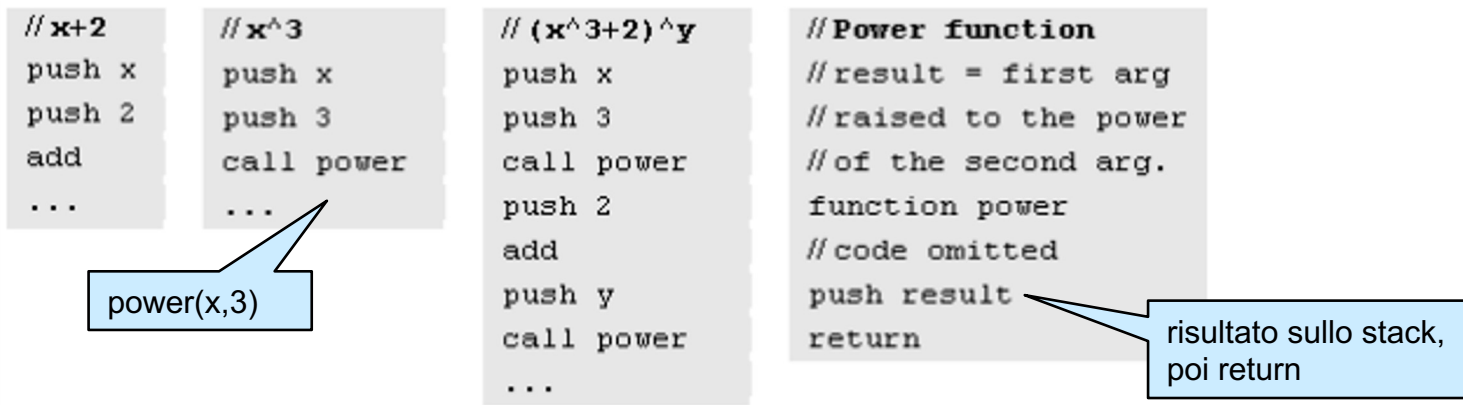
Comandi per il controllo del flusso

Per implementare costrutti del flusso di controllo (if-then-else, while, etc) procediamo come in assembly:

- `label NOME`: dichiara una etichetta NOME
- `goto NOME`: jump incondizionato all'etichetta NOME
- `if-goto NOME`: jump condizionato: si effettua il salto se *true* (in binario in complemento a due: -1) è in cima allo stack. **Esegue anche un pop!**

```
// esempio: if local[0]>3 then local[0]=0 else ...
...
push local 0
push constant 3
gt
not                // la valutazione della condizione negata è in cima allo stack
if-goto ELSE       // salto condizionato
push constant 0
pop local 0
label ELSE
...
// ... si poteva fare anche in altro modo ...
```

Chiamata a funzione (subroutine)



Convenzione di chiamata:

- La funzione chiamante (o *chiamante*) fa push degli argomenti, poi fa la “call” del *chiamato* e aspetta il “return”
- Prima di terminare, il chiamato deve fare push del valore di ritorno
- Al momento della “return” le risorse usate dal chiamato vengono rilasciate ed il chiamante è ripristinato nel medesimo stato in cui era al momento della “call”

Effetto netto: il chiamato consuma gli argomenti e lascia il valore di ritorno in cima allo stack esattamente come succede nelle operazioni di base (add, or,..)

Comandi relativi alle subroutine

Per le chiamate si utilizzano questi comandi:

- `function nomefunzione numVarLocali`

Inizio della definizione della funzione. Nota: deve specificare il **numero di variabili locali** di cui ha bisogno (così che queste vengano allocate in local);

- `call f numArgomenti`

Invoca la funzione f. Nota: specifica **quanti argomenti** sono attesi, così che venga fatto il numero giusto di operazioni pop, e si allochi anche la memoria necessaria nel segmento argument;

- `Return`

Istruzione di ritorno. Il risultato della funzione è già stato inserito in cima allo stack, quindi il controllo può tornare al chiamante.

Il protocollo di call-and-return di funzioni

Un compilatore per VM deve assicurarsi che il programma ISA esegua quanto segue. Questo non è altro che il corrispettivo di quanto studiato in teoria per le chiamate a procedure (qui però ci sono anche i segmenti, quindi dati diversi vanno in segmenti diversi).

Lato chiamante:

- Prima della chiamata, occorre inserire gli argomenti in cima allo stack
- Successivamente si invoca la funzione richiesta con il comando call
- Dopo che la funzione chiamata ritorna:
 - Gli argomenti inseriti sullo stack prima della chiamata sono rimossi; al loro posto è presente in cima allo stack il valore di ritorno (il risultato);
 - Tutti i segmenti di memoria sono esattamente gli stessi di prima della chiamata.

Lato chiamato:

- All'inizio, il segmento argument contiene i parametri attuali passati dal chiamante
- Il segmento local è presente e contiene tutti valori uguali a zero
- Lo stack (porzione visibile a questa funzione) è vuoto (si ottiene modificando lo SP)
- Prima del return si lascia sullo stack il valore di ritorno (e niente altro)

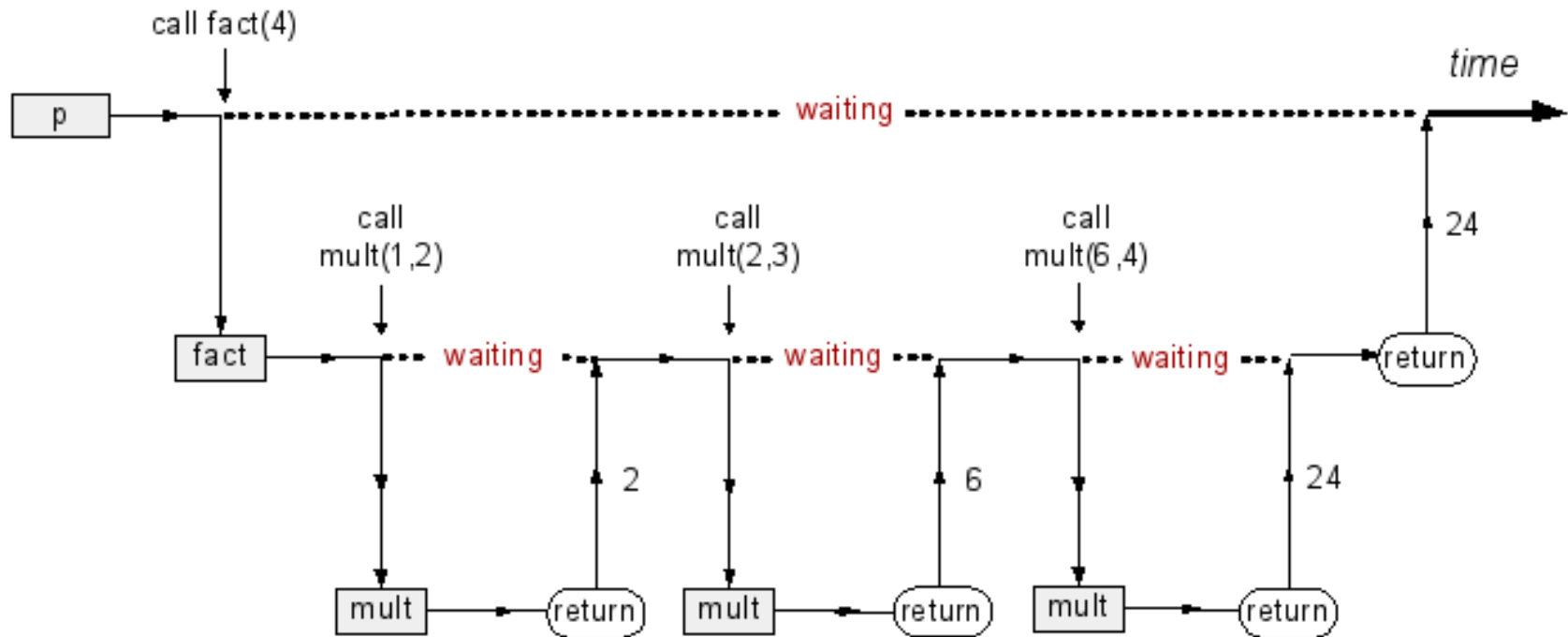
In coda a queste slide, qualche altro dettaglio su come supportare questo con il computer Hack.

Esempio

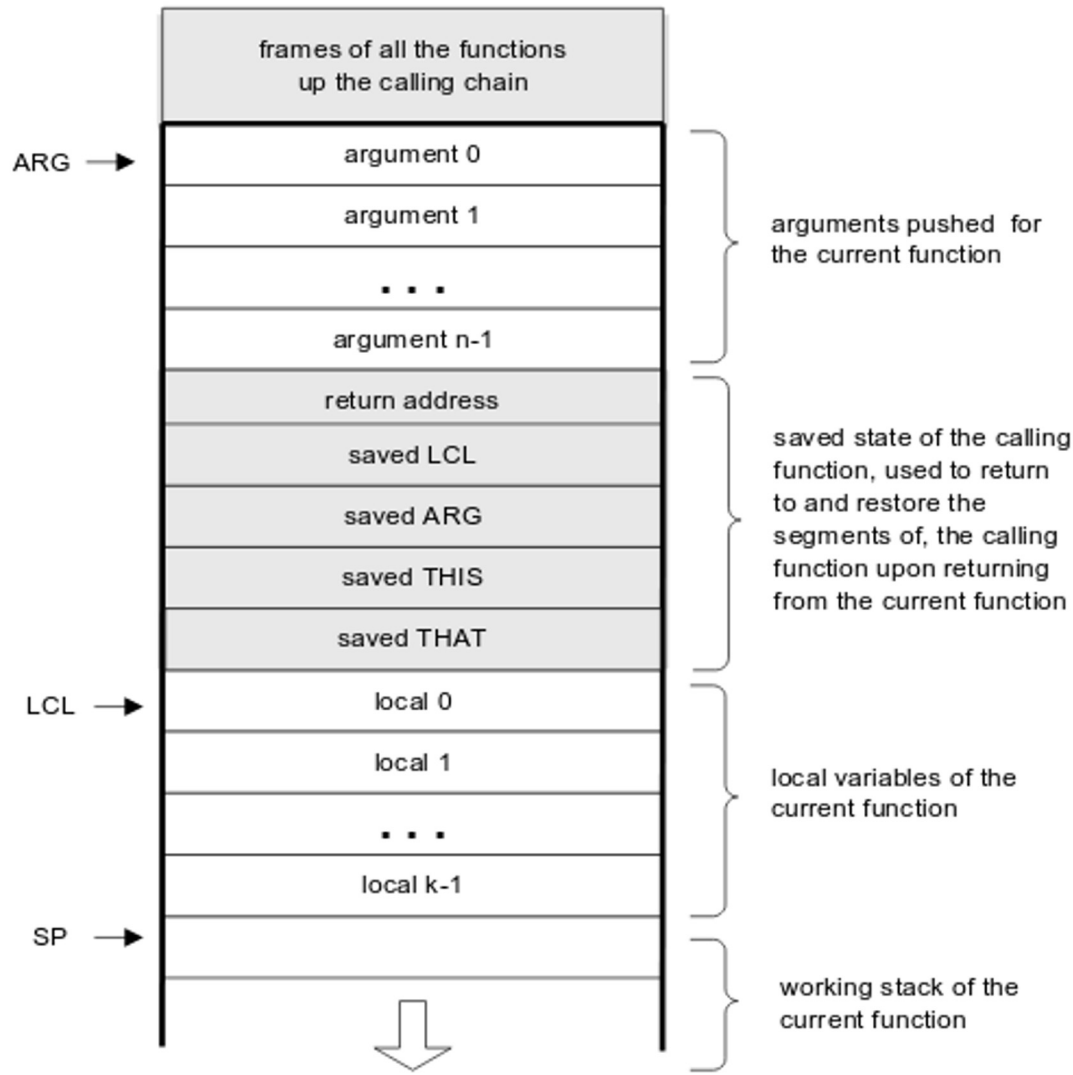
```
function p(...) {  
  ...  
  ... fact(4) ...  
}
```

```
function fact(n) {  
  vars result,j;  
  result=1; j=2;  
  while j<=n {  
    result=mult(result,j);  
    j=j+1;  
  }  
  return result;  
}
```

```
function mult(x,y) {  
  vars sum,j;  
  sum=0; j=y;  
  while j>0 {  
    sum=sum+x;  
    j=j+1;  
  }  
  return sum;  
}
```



Al livello fisico, è tutto implementato su un unico Stack nella memoria di Hack



Usualmente c'è una catena di funzioni in attesa di essere riattivate, ed una sola funzione in esecuzione

Aree grigie: parti irrilevanti per la funzione in esecuzione

La funzione corrente si interessa solo della parte più recente dello stack (in bianco, corrispondente al suo record di attivazione, in figura detto *working stack*).

A differenza di quanto visto in teoria, l'uso di segmenti comporta che il record corrente non occupa locazioni tutte consecutive (vedi per esempio LCL e ARG per variabili locali e argomenti).

SP è la cima, come sappiamo (figura al contrario).

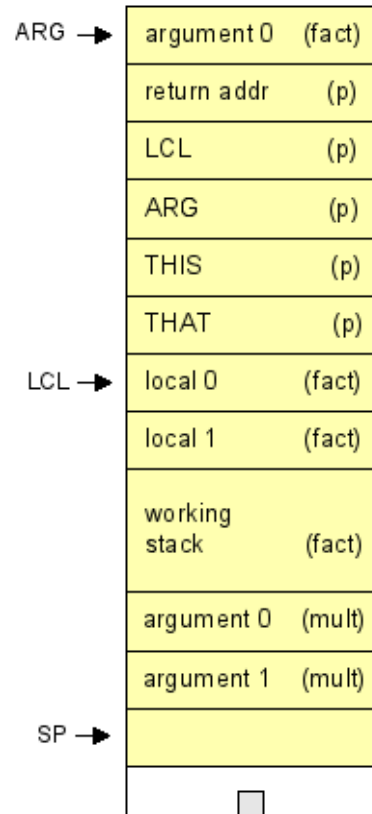
Visto sullo stack..

```
function p(...) {  
  ...  
  ... fact(4) ...  
}
```

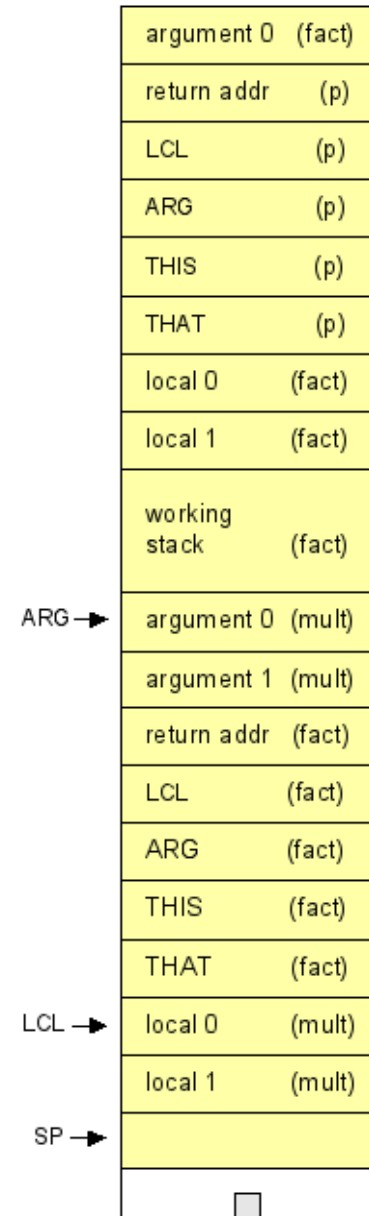
```
function fact(n) {  
  vars result,j;  
  result=1; j=1;  
  while j<=n {  
    result=mult(result,j);  
    j=j+1;  
  }  
  return result;  
}
```

```
function mult(x,y) {  
  vars sum,j;  
  sum=0; j=y;  
  while j>0 {  
    sum=sum+x;  
    j=j+1;  
  }  
  return sum;  
}
```

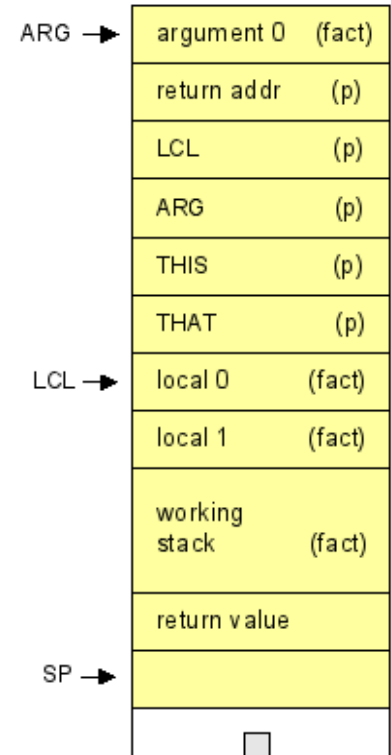
just before "call mult"



just after mult is entered



just after mult returns



Esempio: funzione chiamata: definizione

Pseudocodice (sinistra), codice incompleto senza specifica dei segmenti (centro), codice VM Hack (destra)

```
int myfun(int x)
{
    z = x+1

    if(z>=5)
    {
        return(0);
    }
    else
    {
        return(z);
    }
}
```

```
function myfun
    push x
    push 1
    add
    pop z
    push z
    push 5
    lt
    push 0
    return
label ELSE
    push z
    return
```

```
function myfun 1
    push argument 0
    push constant 1
    add
    pop local 0
    push local 0
    push constant 5
    lt
    push constant 0
    return
label ELSE
    push local 0
    return
```

1 var. locale

x è salvata in argument[0]

salviamo z in local[0]

Il codice parziale (al centro) NON è corretto perché mancano i segmenti. E' mostrato in questa slide per facilitare la comprensione del programma completo.

I nomi delle funzioni dovrebbero essere: nomefile.nomefunzione
Sulle slide e negli esercizi trascuriamo questo dettaglio.

Esempio: funzione chiamante

Non ci soffermeremo in queste slide sul caso del chiamante, perché ne avremo bisogno solamente in caso di svolgimento del progetto. Esempi sono su Virtuale e sito Nand2Tetris.

```
function Main.fibonacci 0
push argument 0
push constant 2
lt
// checks if n<2
if-goto IF_TRUE
goto IF_FALSE
label IF_TRUE
// if n<2, return n
push argument 0
return
label IF_FALSE
// if n>=2, returns fib(n-2)+fib(n-1)
push argument 0
push constant 2
sub
call Main.fibonacci 1 // computes fib(n-2)
push argument 0
push constant 1
sub
call Main.fibonacci 1 // computes fib(n-1)
add
// returns fib(n-1) + fib(n-2)
return
```

```
//computa n-esimo
//numero di Fibonacci
function Sys.init 0
push constant 4
call Main.fibonacci 1
label END
goto END //ciclo dummy
```

Sys.vm

Main.vm

Esercizio

Si scriva codice per la VM Hack che implementa la funzione riportata in pseudocodice.

```
function dummy(x, y)
{
    int z;
    z = 0;

    if(x > y)
        return x;

    else
    {
        z = y + 3;
        return z > x;
    }
}
```

Esercizio: soluzione

Si scriva codice per la VM Hack che implementa la funzione riportata in pseudocodice.

```
function dummy(x, y)
{
    int z;
    z = 0;

    if(x > y)
        return x;

    else
    {
        z = y + 3;
        return z > x;
    }
}
```

```
function dummy 1          // 1 variabile locale
    push constant 0
    pop local z           // z=0
    push argument 0        // x
    push argument 1        // y
    gt
    not
    if-goto ELSE          // se x<=y salta a label ELSE
    push argument 0
    return                 // return x
label ELSE
    push argument 1
    push constant 3
    add
    pop local 0            // z=y+3
    push local 0
    push argument 0
    gt
    return                 // ritorna il valore in cima

//possiamo includere un ciclo infinito, come in assembly
```


Esercizio

Si scriva codice per la VM Hack che implementa la funzione riportata in pseudocodice.

```
int fun(int a)
{
    while (a<15)
    {
        a+=4;
    }
    return(-a);
}
```

Esercizio: soluzione

Si scriva codice per la VM Hack che implementa la funzione riportata in pseudocodice.

```
function mult(x,y)
{
  int sum, j;
  sum = 0;
  j = y;
  while (j != 0)
  {
    sum = sum + x;
    j = j - 1;
  }
  return sum;
}
```

```
function mult 2          // 2 variabili locali
  push  constant 0
  pop   local 0          // sum = 0
  push  argument 1      // y
  pop   local 1          // j = y
label LOOP
  push  local 1
  push  constant 0
  eq
  if-goto END
  push  local 0
  push  argument 0
  add
  pop   local 0
  push  local 1
  push  constant 1
  sub
  pop   local 1
  goto LOOP
label END
  push  local 0
  return
```

Esercizio

Si scriva codice per la VM Hack che implementa la funzione riportata in pseudocodice.

```
int f(int x, int y)
{
    int z;
    z = 3;

    while(x>0)
    {
        y+=z;
        x-=2;
    }
    return x;
}
```

Esercizio: soluzione

Si scriva codice per la VM Hack che implementa la funzione riportata in pseudocodice.

```
int f(int x, int y)
{
    int z;
    z = 3;

    while(x>0)
    {
        y+=z;
        x-=2;
    }
    return x;
}
```

```
function f 1
    push constant 3
    pop local 0
label LOOP
    push argument 0
    push constant 0
    gt
    not
    if-goto EXIT
    push local 0
    push argument 1
    add
    pop argument 1
    push argument 0
    push constant 2
    sub
    pop argument 0
    goto LOOP
label EXIT
    push argument 0
    return
```

Esercizio:

Si scriva codice per la VM Hack che implementa la funzione riportata in pseudocodice.

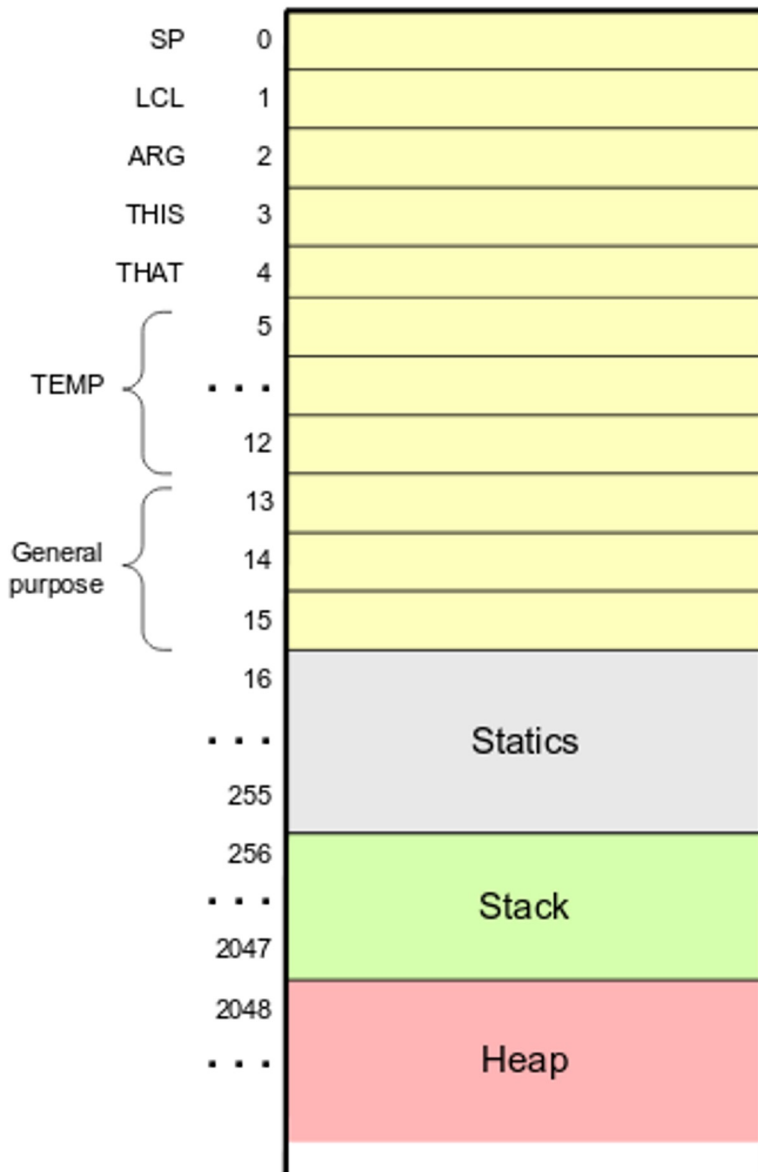
```
int myfunction(int x)
{
    int sum=100;

    while(x<=sum)
    {
        x=x*2;
        sum=sum-2;
    }

    return((x-sum)>10);
}
```

Alcuni dettagli per il progetto 6

Implementazione della memoria sul computer Hack



Dal punto di vista “fisico”, per eseguire un programma per VM Hack, la memoria fisica del computer Hack viene organizzata nel seguente modo:

- Gli indirizzi da 0 a 15 non saranno parte di specifici segmenti, ma saranno usati a livello di implementazione della virtual machine
- In particolare, noi useremo SP (stack pointer), LCL (local segment pointer), ARG (argument segment pointer) (gli altri indirizzi fino a 12 sono usati in una implementazione più completa nel progetto nand2tetris, da 13 a 15 lasciati liberi)
- Gli indirizzi da 16 a 255 saranno usati per il segmento “static” (l’accesso a questo segmento serve all’assembler: ricorda i simboli di variabile).
- Gli indirizzi da 256 a 2047 saranno usati per lo stack (operazioni push e pop) e per i segmenti “argument” e “local” creati al momento dell’invocazione di funzioni (SP inizializzato a 256)
- Heap non serve per questo progetto

Note su VMEmulator

- Il nome delle funzioni deve essere del tipo `nomefile.nomefunzione` (dal `nomefile` rimuovere l'estensione)
- Come già ricordato, il VMEmulator non inizializza i segmenti local (LCL) e argument (ARG) scrivendo in `RAM[1]` e `RAM[2]`. Questi vengono inizializzati se fate una chiamata di funzione, mentre per testare una funzione senza una effettiva chiamata, bisogna settarli manualmente (per esempio con un file `.tst` come quello chiamato `test.tst` su Virtuale). Lo SP si inizializza a 256 in `RAM[0]`.

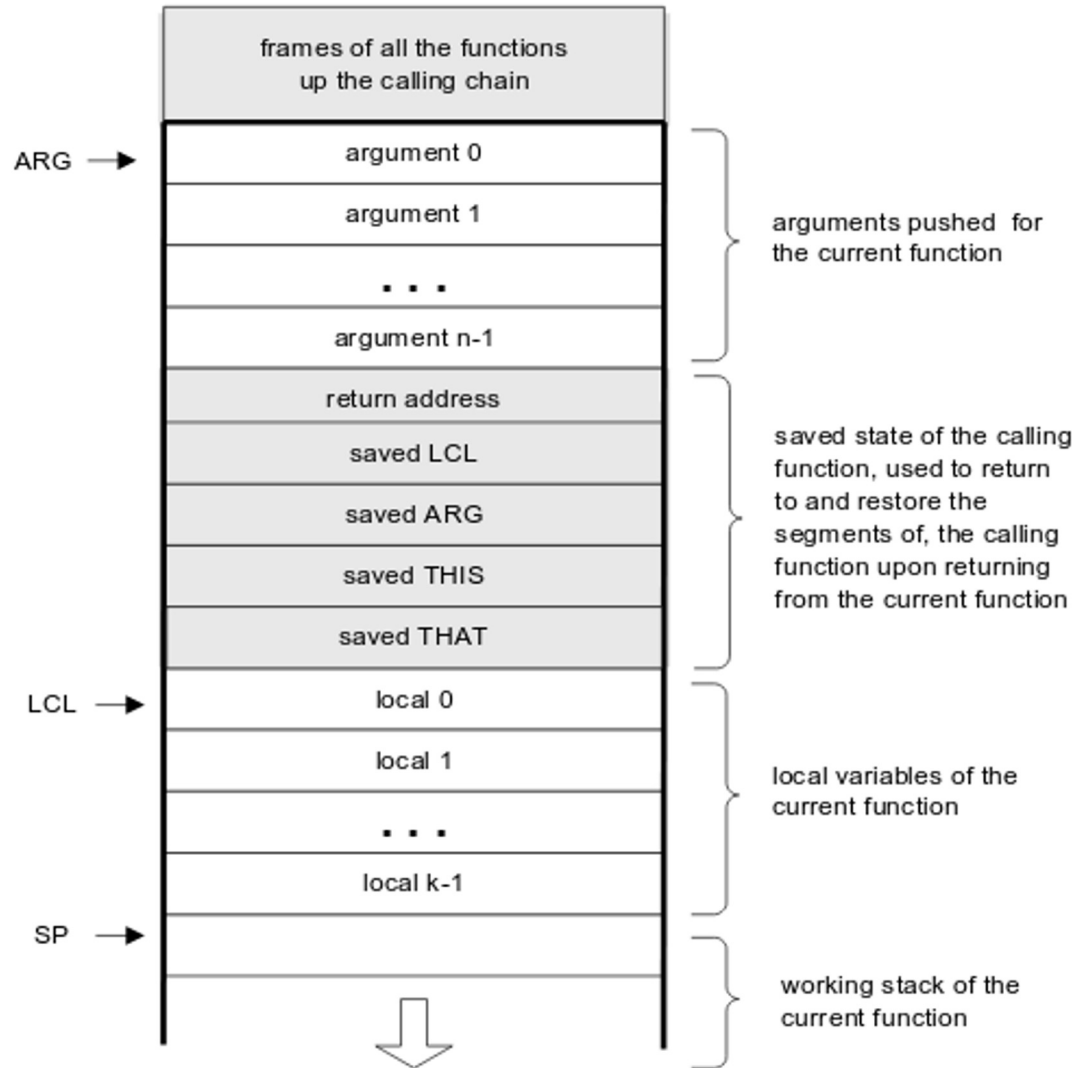
Dettagli per l'implementazione del traduttore VM -> assembly

- Traduzione dei comandi relative al flusso di controllo: semplicemente usare il meccanismo delle dichiarazioni di label e dei jump di assembly

Per le funzioni:

- Attenzione al rilascio delle risorse del chiamato ed al ripristino dello stato del chiamante. Questa è la parte più critica dell'implementazione
- Quando la funzione *f* chiama la funzione *g*, l'implementazione della VM deve:
 - Salvare il "return address" (indirizzo in ROM della istruzione successiva)
 - Salvare i puntatori ai segmenti local e argument di *f*
 - Impostare i puntatori local e argument di *g*
 - Passare il controllo all'implementazione di *g*
- Quando la funzione *g* inizia ad eseguire, l'implementazione della VM deve: Predisporre lo spazio per il segmento local (inizializzando tutte le sue celle a 0)
- Quando *g* termina ed il controllo deve ritornare a *f*, l'implementazione della VM deve: Eliminare i segmenti local e argument lasciando sullo stack solamente il valore di ritorno; Ripristinare i segmenti local e argument di *f*; Restituire il controllo a *f* saltando al "return address"

Al livello fisico, è tutto implementato su un unico Stack nella memoria di Hack



Implementazione del comando: call f n

call f n

(calling a function f after n arguments have been pushed onto the stack)

vedi figura in slide
precedente

```
push return-address // (Using the label declared below)
push LCL             // Save LCL of the calling function
push ARG             // Save ARG of the calling function
push THIS            // Save THIS of the calling function
push THAT            // Save THAT of the calling function
ARG = SP-n-5         // Reposition ARG (n = number of args)
LCL = SP             // Reposition LCL
goto f               // Transfer control
(return-address)     // Declare a label for the return-address
```

Questa è la versione completa del progetto nand2tetris: a noi bastano i segmenti local e argument (quindi non sarebbe per noi necessario considerare THIS e THAT)

Implementazione del comando: function f k

function f k

(declaring a function f that has k local variables)

```
(f)                // Declare a label for the function entry
  repeat k times:   // k = number of local variables
    PUSH 0           // Initialize all of them to 0
```

Implementazione del comando: return

return

(from a function)

```
FRAME=LCL           // FRAME is a temporary variable
RET=* (FRAME-5)      // Put the return-address in a temp. variable
*ARG=pop ()          // Reposition the return value for the caller
SP=ARG+1             // Restore SP of the caller
THAT=* (FRAME-1)      // Restore THAT of the caller
THIS=* (FRAME-2)      // Restore THIS of the caller
ARG=* (FRAME-3)       // Restore ARG of the caller
LCL=* (FRAME-4)       // Restore LCL of the caller
goto RET             // Goto return-address (in the caller's code)
```

Questa è la versione completa del progetto nand2tetris: a noi bastano i segmenti local e argument (quindi non sarebbe per noi necessario considerare THIS e THAT)